Roadmap: Building a Windows Productivity Assistant (Solo)

This is a **no-excuses, execution-first roadmap**. The goal is not a perfect product. The goal is to **build a real Windows background assistant** that does useful work, then evolve it into something intelligent.

PHASE 0 — MENTAL RESET (1 DAY)

Objective: Kill confusion and lock focus.

- Stop researching new languages/tools
- Freeze stack:
- Go → core logic & Windows service
- PostgreSQL (later) → persistence
- Python (later) → AI only
- Accept this truth: *Version 1 will be ugly and dumb*

Deliverable: - One-page project description in your own words - Clear statement: *"This is a background Windows service that manages tasks and focus"*

PHASE 1 — WINDOWS SERVICE FOUNDATIONS (WEEKS 1–2)

Objective: Become dangerous with Windows services in Go.

Step 1: Environment Setup

- Windows 10/11 dev machine
- Install Go (latest stable)
- Set up Git repo (local first)

Step 2: Learn Windows Service Basics

- Understand:
- Service Control Manager (SCM)
- Service lifecycle (start/stop/shutdown)
- Read Microsoft docs (concepts only)

Step 3: First Go Service

- Use `golang.org/x/sys/windows/svc`
- Implement:
- Service install/uninstall

- Start/stop handling
- Logging to a file

Deliverable: - A Go binary that: - Installs as a Windows service - Starts on boot - Logs a heartbeat every 5 seconds

---

PHASE 2 — CORE ENGINE (WEEKS 3–4)

Objective: Build the brain without AI.

Step 4: Task Model

- Define task structure:
- ID
- Title
- Trigger time
- Action type

Step 5: Scheduler Loop

- Background goroutine
- Checks tasks every second
- Triggers actions on time

Step 6: Basic Actions

Implement at least 2: - System notification - File creation (e.g., create a .txt or .docx)

Deliverable: - Service executes scheduled actions reliably

PHASE 3 — LOCAL CONTROL INTERFACE (WEEKS 5–6)

Objective: Control the service without touching the service.

Step 7: IPC Mechanism

Choose ONE: - Local HTTP API (localhost only) - Named pipes (harder, optional)

Step 8: CLI Tool

- Create a small CLI app in Go
- Commands:
- add-task
- list-tasks
- delete-task

Deliverable: - You can fully control the service from terminal

---

PHASE 4 — FOCUS MODE & DISCIPLINE (WEEKS 7–8)

Objective: Real value feature.

Step 9: Focus Mode

- Define focus session
- During focus:
- Block specific apps (crude methods allowed)
- Silence notifications

Step 10: Logging & Audit

- Log:
- Task execution
- Focus sessions
- Failures

Deliverable: - A tool that actively protects user time

PHASE 5 — DATA & POLISH (WEEKS 9–10)

Objective: Make it survivable.

Step 11: Persistence

- Add PostgreSQL OR SQLite
- Store:
- Tasks
- Execution history

Step 12: Reliability

- Graceful shutdown
- Panic recovery
- Restart-safe scheduling

Deliverable: - Service survives reboots and crashes

PHASE 6 — AI INTRODUCTION (OPTIONAL, LATER)

Objective: Earn the right to use AI.

Step 13: Natural Language Parsing

- Use Python
- Simple model or API
- Convert text → structured task

Example:

> "Remind me to email Alex tomorrow at 9"

Step 14: Integration

- Go service calls Python module
- Receives structured task

Deliverable: - Text-based AI task creation

PHASE 7 — VOICE & EXPANSION (FUTURE)

Objective: Only if everything above works.

- Voice input
- Smarter automation
- Third-party integrations

FINAL RULES (NON-NEGOTIABLE)

- One phase at a time
- No skipping
- No tool-hopping
- Working > pretty

If you complete Phase 4, you are no longer a beginner. You are a builder.